ODSB Analysis Report

Ticket: ODSB-17637 | **Generated:** 2026-04-14 01:56 UTC

[NETMARBLE][Deep-dive (campaign goal specific) - Account / Client - ROAS] [NETMARBLE] ROAS campaign not generating revenue

---

Investigation Report: ODSB-17637

> **[WARN] DISCLAIMER:** This report and its data may not be accurate. The information presented could contain errors, incomplete data, or sensitive information that may not be appropriate for public disclosure. Reproducible queries are included so that findings can be validated independently. Please verify before sharing externally.

Alert Type: Low ROAS
Platform: NETMARBLE
Entities: campaign_id=iyH1zVvUZpViudzo, ad_account=yfg0At8VksGnt6EO, product=wxAmPvDyZWpy4yeQ
Date Range: 2026-04-06 to 2026-04-13
Investigation Date: 2026-04-14

Ticket Summary

NETMARBLE ROAS campaign (iyH1zVvUZpViudzo) for "The Seven Deadly Sins Origin_AOS" (com.netmarble.nanaori) is barely generating revenue. Campaign launched April 6, 2026. D7 ROAS = 1.05% after 8 days.

Alert Criteria:
D7 ROAS critically low at 1.05% after 8 days of campaign operation, indicating the campaign is returning approximately $0.01 for every $1 spent.

Root Cause

The campaign's ROAS model is in cold-start phase — at only 8 days old, the model lacks sufficient data to calibrate properly, resulting in bid prices 10-50x below exchange floor prices.

Confidence: HIGH (cross-confirmed across 6 independent evidence sources)

Impact: $6,195 spent with only $65.02 D7 revenue (1.05% ROAS). 99.8% of bid candidates are rejected by FilterByBidfloor because bid CPMs ($0.05-$0.08) are far below exchange floors ($0.50-$2.00). The revenue model's normalizer is frozen, the pred calibrator has not yet activated (requires 15 days), and the I2I model over-predicts installs by 2-4x.

Charts (Google Drive)

- Daily Performance, TCM Trend, I2I Calibration, Exchange Spend
- Bid CPM Distribution: FilterByBidfloor vs CommitBid

Supporting Data (Google Sheets)

- Daily Spending Trend
- NHN Aggregated Filter Funnel (no reason_raw) - 8 Days
- Entity Change History - Budget and Goal Changes
- Install Model Calibration (I2I)
- Revenue Event Details
- Campaign iyH1zVvUZpViudzo Daily Performance (Apr 6-13 2026)
- Model Multipliers from Imp Stream

- Pricing Pass Rate by Exchange (310 vs 305)
- tKPI Multipliers by Candidate Result
- Cross-Confirm: I2I Calibration Comparison
- F4: Payer Analysis - Revenue Distribution per Device
- F3: Revenue Events by Campaign for com.netmarble.nanaori
- RTB Funnel Comparison - NHN vs Top Exchanges
- Bid Funnel by Exchange (Top 10) - Full 8 Days
- RTB Funnel by Exchange - fact_supply - 8 Days
- CommitBid Bid Details
- Budget History
- Top 5 Exchanges Daily Detail - 8 Days
- Exchange Performance Metrics - 8 Days
- NHN Daily Trend - 8 Days
- Daily Performance with Revenue
- All campaigns for com.netmarble.nanaori (Apr 6-13 2026)
- Overall Campaign Filter Funnel - 8 Days
- Daily Revenue Summary
- Core and Wrapper Multipliers for Auction Participants
- Campaign Configuration (Latest Snapshot)
- NETMARBLE ROAS-Only Campaign Comparison (Apr 6-13 2026, spend>100)
- Cross-Confirm: Model Predictions Comparison
- Revenue Postback Summary
- F3c: Cross-Campaign Revenue Efficiency Comparison
- Performance by Creative Format
- NHN Pricing Filter (305) Reason Raw Breakdown

- Pricing Filter (305) Comparison Across Top Exchanges
- Attribution Breakdown
- NHN Full Filter Funnel Detail - 8 Days
- Wrapper Multiplier by Model Type
- Exchange Pricing Comparison - 8 Days
- Internal Auction Funnel Rates
- Entity Change History - Key Changes Only
- Model Predictions and Normalizer Trend
- Daily Performance Metrics
- NETMARBLE Campaign Benchmark Comparison (Apr 6-13 2026)
- Daily Internal Auction Funnel
- NETMARBLE ROAS Campaigns Benchmark with Revenue (Apr 6-13 2026)
- MEMS API Audit Trail - WHO Changed WHAT
- dim2_campaign Daily Snapshots
- MMP Integration Check
- Daily Install vs Revenue Events
- Campaign Configuration
- Cross-Confirm: A2R Calibration Comparison
- Daily Model Predictions and Normalizers
- F3b: All Revenue Events (attributed + unattributed) for App
- NHN Exchange Detail by Format - 8 Days
- Ad Group uqRAOdUWb36lzqdy Configuration (Latest)
- Exchange Funnel Summary - 8 Days
- Revenue Timing: Install-to-First-Revenue Delay
- Bid Price Analysis (1pct sample)

- Candidate Result Distribution
- Cross-Confirm: Multiplier & Bid Comparison
- Ad Group Changes Since April 1
- Entity Changes Since April 1 2026
- Prediction Logs for CommitBid Winners
- Full Exchange Performance (fact_dsp_all gross_spend) - 8 Days
- FilterByBidfloor Analysis
- Revenue Timing: Distribution by Time Bucket
- dim2_campaign Daily Snapshots
- All com.netmarble.nanaori Campaigns (Apr 6-13 2026)
- Daily TCM and Internal CPM
- Performance by Exchange
- NHN Non-Pricing Filter Detail (055,110,218)
- TCM and Budget Pacing Trend

Timeline

Time (UTC)	Event
2026-04-06	Campaign launched with ROAS goal type
2026-04-06 to 2026-04-08	TCM rises from 1.3 to 2.5 as system attempts to spend budget
2026-04-06 to 2026-04-13	A2R normalizer frozen at 15.242939 (zero variance, p1=p99)
2026-04-06 to 2026-04-13	Daily budget changed 5+ times ($980 -> $2107 -> $230 -> $500 -> $980 -> $653)
2026-04-09 to 2026-04-11	TCM accelerates from 3.4 to 5.8 — model increasingly desperate to spend
2026-04-13	TCM reaches 8.6x; bid CPMs still only $0.05-$0.08 vs $0.50-$2.00 floors
2026-04-13	**D7 ROAS = 1.05% after 8 days — alert triggered**

Time (UTC)	Event
2026-04-21 (projected)	Day 15: CALIBRATOR_ACTIVATION_WINDOW reached, pred calibrator activates

---

Evidence Chain

1. A2R (Revenue) Normalizer is Frozen

The revenue normalizer value 15.242939 has zero variance across all 8 days (p1 = p99 = 15.242939). Revenue model normalizers are baked into TFServing as a StaticHashTable and only update on model retraining — there is NO daily update mechanism for revenue normalizers (unlike action models).

In contrast, the mature stonekey campaign (same advertiser, same goal, KOR Android) has an A2R normalizer of 29.93 (nearly 2x higher), reflecting more accurate revenue signal.

Code: `tfserving.go:339-358` — `parseRevenueScore()` revenue normalizer selection priority logic

Code: `revenue_normalizer_utils.py` — Revenue normalizer as StaticHashTable

2. Pred Calibrator Not Activated

The `CALIBRATOR_ACTIVATION_WINDOW = 15` days in `constants.py`. This campaign is only 8 days old, so the calibration multiplier is NULL across all 71,350 sampled candidates. Without calibration, model predictions remain uncorrected.

Code: `constants.py` — CALIBRATOR_ACTIVATION_WINDOW=15, daily clip bounds

3. Zero A2R Calibration Data

`validation_a2r_online` has ZERO rows for this campaign. The revenue model has no signal to learn from.

Comparison: the stonekey campaign (nazpxG3J5MareHRz, 6 weeks old) has A2R calibration ratios ranging 0.56-1.29, indicating the model has meaningful revenue data to calibrate against.

4. I2I Model Over-Predicting 2-4x

The install model calibration ratio ranges 0.27-0.71, meaning the model predicts 2-4x more installs than actually occur. This inflates bid values in theory but is dwarfed by the frozen/missing revenue components.

Comparison: stonekey I2I calibration ratio 0.69-1.54 (well-calibrated, centered around 1.0).

5. Multiplier Trap

TCM (budget pacing multiplier) rose from 1.3 to 8.6 as the system desperately tries to spend the daily budget. However, with I2I predictions around 0.00005 and uncalibrated revenue predictions, final bid CPMs are only $0.05-$0.08 — far below exchange floors of $0.50-$2.00. Result: 99.8% of candidates are rejected by FilterByBidfloor.

Note per lessons learned: TCM is a budget pacing multiplier, NOT a throttling indicator. The high TCM here reflects under-spending, not over-spending.

6. Exchange Mix Inefficiency

NHN exchange consumes 31% of total spend ($1,951) but delivers only 9 installs at $217 CPI with ZERO revenue. NHN bidfloors are $1.35-$2.00 CPM with 99.98% banner format. The cold model cannot steer spend away from inefficient exchanges.

Charts

(See Figure 1: Daily Performance, TCM Trend, I2I Calibration, Exchange Spend)

(See Figure 2: Bid CPM Distribution: FilterByBidfloor vs CommitBid)

Comparison: nanaori vs stonekey (same advertiser, same goal, KOR Android)

Metric	nanaori (8 days)	stonekey (6 weeks)
D7 ROAS	1.05%	9.88%

Metric	nanaori (8 days)	stonekey (6 weeks)
CPI	$54.95	$48.38
I2I calibration ratio	0.27-0.71	0.69-1.54
A2R calibration data	ZERO rows	Has data (0.56-1.29)
A2R normalizer	15.24 (frozen)	29.93 (2x higher)
Revenue wrapper multiplier	1.68-2.42 (inflated)	1.11-1.32 (well-calibrated)
ACTION normalizer	0.017-0.028	0.089-0.117

Campaign Performance Summary (8 days)

Metric	Value
Impressions	6.31M
Clicks	20,908
Installs	113
Revenue (D7)	$65.02
Spend	$6,195
D7 ROAS	1.05%
CPI	$54.95
IPM	0.018
CTR	0.33%
CVR	0.54%
Unique payers	7 (6.2% payer conversion)

Metric	Value
Revenue per install	$0.58

Attribution/MMP Status

- 100% CT (click-through), 0% VT — no attribution anomalies
- 100% attributed to Moloco
- campaign_id populated for 100% of events
- AppsFlyer `revenue` event working correctly (17 events, $65.02)
- No tracking/attribution issues detected

Code References

Code Path	Lines	Description	GitHub Link
`go/src/bidfnt/pricingmodel/predictors/prob/tfserving.go`	339-358	`parseRevenueScore()` — revenue normalizer selection priority logic	View on GitHub
`python/moloco-training/moloco/training/utilities/revenue_normalizer_utils.py`	Full	Revenue normalizer as StaticHashTable	View on GitHub
`python/moloco-training/moloco/training/scripts/revenue_lat/install_to_revenue/prepare_preproc.py`	755-815	Normalizer lookup table creation	View on GitHub
`python3/ops_tasks/pricingmodel/jobs/ua_revenue.py`	Full	Revenue pricing metadata updater (Airflow DAG)	View on GitHub
`python/moloco-scripts/moloco/scripts/ml_pred_calibrator_generation/constants.py`	Full	CALIBRATOR_ACTIVATION_WINDOW=15, daily clip bounds	View on GitHub
	Full	Pred calibrator generation algorithm	

Code Path	Lines	Description	GitHub Link
`python/moloco-scripts/moloco/scripts/ml_pred_calibrator_generation/ml_pred_based_calibrator_update.py`			View on GitHub
`go/src/bidfnt/pricingmodel/functions/closure.go`	548-569	tKPI budget multiplier formula	View on GitHub
`go/src/bidfnt/entity/bidcontext.go`	1687-1702	`GetPredCalibrator()` — calibrator selection	View on GitHub

Questions Resolved

#	Question	Status	Finding
1	Why is D7 ROAS only 1.05%?	RESOLVED	Cold-start model: revenue normalizer frozen, pred calibrator not activated (needs 15 days), zero A2R calibration data
2	Is there a tracking/attribution issue?	RESOLVED	No — 100% CT attribution, revenue events flowing correctly via AppsFlyer
3	Is the I2I model well-calibrated?	RESOLVED	No — over-predicts by 2-4x (calibration ratio 0.27-0.71), but this is secondary to the revenue model issues
4	Why is spend being consumed despite poor ROAS?	RESOLVED	TCM (budget multiplier) escalated from 1.3 to 8.6x to force spending; only the tiny fraction of bids above floor prices win
5	Is NHN exchange efficient?	RESOLVED	No — 31% of spend ($1,951), 9 installs ($217 CPI), zero revenue. Cold model cannot steer away
6	How does this compare to a mature campaign?	RESOLVED	stonekey (same advertiser, 6 weeks old) has 9.88% ROAS, calibrated models, and A2R data — confirming cold-start as the differentiator

---

Alternative Explanations Ruled Out

Alternative	Why Ruled Out
Tracking/Attribution issue	100% CT attribution, campaign_id populated for all events, revenue events flowing correctly (17 events, $65.02)
MMP misconfiguration	AppsFlyer revenue events are being received and attributed correctly
Budget too low	Budget ranged $230-$2,107/day; spend was $6,195 over 8 days — budget is not the constraint
Creative quality issue	CTR (0.33%) and CVR (0.54%) are low but not abnormal for a cold campaign; the funnel issue is pre-bid (FilterByBidfloor), not post-impression
Exchange-side blocking	Bids are being submitted but rejected due to low bid price, not exchange blocking
App monetization issue	7 unique payers generated $65.02; the app does monetize, but low install volume means low revenue base

---

Recommendations

1. **Allow more time for model warm-up** — ROAS models need 4+ weeks to stabilize. The pred_calibrator will activate after day 15 (CALIBRATOR_ACTIVATION_WINDOW). By week 3-4, model predictions should improve significantly as the A2R normalizer gets updated on model retraining.

2. **Consider running CPI phase first** — A CPI campaign would accumulate install and revenue data faster, giving the ROAS model signal to learn from before switching to ROAS optimization. The same-app CPA campaign (HTVA26OzfthK6LPa) achieves CPI=$6.43 and has 302 unique payers — demonstrating the app can acquire users efficiently.

3. **Review NHN exchange allocation** — NHN is burning 31% of budget ($1,951) at $217 CPI with zero revenue. Consider excluding NHN temporarily or capping spend until model is warmer.

4. **Set realistic expectations** — With 113 installs and 7 payers, ROAS is statistically unreliable and heavily whale-dependent (1 payer = 57% of revenue). Need 500+ installs for meaningful ROAS measurement.

5. **Stabilize budget during warm-up** — Daily budget changed 5+ times in 8 days ($980 -> $2107 -> $230 -> $500 -> $980 -> $653). Frequent budget changes hinder model learning. Recommend holding budget steady during the warm-up period.

Reproducible Queries

Query 1: Campaign Configuration (Entity History)

```
SELECT
  timestamp,
  JSON_VALUE(json_entity, '$.name') AS campaign_id,
  JSON_VALUE(json_entity, '$.display_name') AS display_name,
  JSON_VALUE(json_entity, '$.enable') AS enabled,
  JSON_VALUE(json_entity, '$.country') AS country,
  JSON_VALUE(json_entity, '$.os') AS os,
  JSON_VALUE(json_entity, '$.capper.user_budget_enforcer.daily_budget') AS daily_budget,
  JSON_VALUE(json_entity, '$.campaign_goal_model.type') AS goal_type,
  JSON_QUERY(json_entity, '$.campaign_goal_model') AS goal_model_full
FROM `focal-elf-631.entity_history.prod_entity_history`
WHERE DATE(timestamp) >= '2026-04-01'
  AND entity_type = 'RTB_CAMPAIGN'
  AND JSON_VALUE(json_entity, '$.name') = 'iyH1zVvUZpViudzo'
ORDER BY timestamp DESC
LIMIT 1
```

Query 2: Daily Performance Metrics

```
SELECT
  date_utc,
  SUM(impressions) AS impressions,
  SUM(clicks) AS clicks,
  SUM(installs) AS installs,
  SUM(kpi_actions) AS kpi_actions,
  SUM(revenue_d1) AS revenue_d1,
  SUM(revenue_d7) AS revenue_d7,
  SUM(payers_d1) AS payers_d1,
```

```
  SUM(payers_d7) AS payers_d7,
  SUM(gross_spend_usd) AS gross_spend_usd,
  SAFE_DIVIDE(SUM(gross_spend_usd), SUM(installs)) AS cpi,
  SAFE_DIVIDE(SUM(clicks), SUM(impressions)) * 100 AS ctr,
  SAFE_DIVIDE(SUM(installs), SUM(clicks)) * 100 AS cvr,
  SAFE_DIVIDE(SUM(installs), SUM(impressions)) * 1000 AS ipm,
  SAFE_DIVIDE(SUM(revenue_d7), SUM(gross_spend_usd)) AS roas_d7
FROM `ads-bpd-guard-china.athena.fact_dsp_core`
WHERE campaign_id = 'iyH1zVvUZpViudzo'
  AND date_utc BETWEEN '2026-04-06' AND '2026-04-13'
GROUP BY date_utc
ORDER BY date_utc
```

Query 3: Daily Spending Trend with Budget

```
WITH daily_spend AS (
  SELECT
    date_utc,
    ROUND(SUM(gross_spend_usd), 2) as gross_spend,
    SUM(impressions) as impressions,
    SUM(installs) as installs
  FROM `moloco-ae-view.athena.fact_dsp_core`
  WHERE date_utc BETWEEN '2026-04-06' AND '2026-04-13'
    AND campaign_id = 'iyH1zVvUZpViudzo'
  GROUP BY 1
),
budget_config AS (
  SELECT
    date_utc,
    daily_budget_local as budget
  FROM `moloco-ae-view.athena.dim2_campaign`
  WHERE campaign_id = 'iyH1zVvUZpViudzo'
    AND date_utc BETWEEN '2026-04-06' AND '2026-04-13'
)
SELECT
```

```
  d.date_utc,
  d.gross_spend,
  b.budget as daily_budget,
  ROUND(d.gross_spend / NULLIF(b.budget, 0) * 100, 1) as budget_util_pct
FROM daily_spend d
LEFT JOIN budget_config b ON d.date_utc = b.date_utc
ORDER BY d.date_utc
```

Query 4: TCM and Budget Pacing Trend

```
SELECT
  date,
  ROUND(AVG(tcm), 4) as avg_tcm,
  ROUND(AVG(budget), 2) as avg_budget,
  ROUND(SUM(spent), 2) as total_spent,
  SUM(imp_cnt) as total_imps,
  SUM(internal_winner_cnt) as total_internal_wins,
  SUM(prc_cnt) as total_prc_cnt
FROM `moloco-data-prod.younghan.spending_summary`
WHERE campaign = 'iyH1zVvUZpViudzo'
  AND date BETWEEN '2026-04-06' AND '2026-04-13'
GROUP BY 1
ORDER BY 1
```

Query 5: Model Predictions and Normalizer Trend

```
SELECT
  date,
  ROUND(AVG(core_pred), 8) as avg_core_pred,
  ROUND(AVG(wrapper_pred), 8) as avg_wrapper_pred,
  ROUND(AVG(wrapper_normalizer), 8) as avg_normalizer,
  ROUND(AVG(a2r_pred), 8) as avg_a2r_pred,
  ROUND(AVG(a2r_normalizer), 8) as avg_a2r_normalizer
```

```
FROM `moloco-data-prod.younghan.spending_summary`
WHERE campaign = 'iyH1zVvUZpViudzo'
  AND date BETWEEN '2026-04-06' AND '2026-04-13'
GROUP BY 1
ORDER BY 1
```

Query 6: Install Model Calibration (I2I)

```
SELECT
  date_imp,
  campaign,
  SUM(cnt_imp) as total_imps,
  SUM(cnt_install) as total_installs,
  SUM(predicted_install) as total_predicted,
  ROUND(SAFE_DIVIDE(SUM(cnt_install), SUM(predicted_install)), 4) as calibration_ratio
FROM `moloco-data-prod.ml_calibration_check.validation_i2i`
WHERE campaign = 'iyH1zVvUZpViudzo'
  AND date_imp BETWEEN '2026-04-06' AND '2026-04-13'
GROUP BY 1, 2
ORDER BY 1
```

Query 7: Revenue Postback Summary

```
SELECT
  DATE(timestamp) AS date_utc,
  LOWER(event.name) AS event_name,
  COUNT(*) AS event_count,
  COUNTIF(moloco.attributed = true) AS attributed_count,
  SUM(event.revenue_usd.amount) AS total_revenue_usd
FROM `focal-elf-631.prod_stream_view.pb`
WHERE DATE(timestamp) BETWEEN '2026-04-06' AND '2026-04-13'
  AND moloco.campaign_id = 'iyH1zVvUZpViudzo'
```

```
GROUP BY 1, 2
ORDER BY 1, event_count DESC
```

Query 8: Attribution Breakdown (CT/VT)

```
SELECT
  DATE(timestamp) AS date_utc,
  COUNTIF(event.click_at IS NOT NULL AND LOWER(event.name) = 'install') AS ct_installs,
  COUNTIF(event.click_at IS NULL AND LOWER(event.name) = 'install') AS vt_installs,
  COUNTIF(LOWER(event.name) = 'install') AS total_installs,
  COUNTIF(moloco.attributed = true AND LOWER(event.name) = 'install') AS attributed_installs
FROM `focal-elf-631.prod_stream_view.pb`
WHERE DATE(timestamp) BETWEEN '2026-04-06' AND '2026-04-13'
  AND moloco.campaign_id = 'iyH1zVvUZpViudzo'
GROUP BY 1
ORDER BY 1
```

Query 9: Revenue Event Details by Type

```
SELECT
  LOWER(event.name) AS event_name,
  COUNT(*) AS event_count,
  COUNTIF(event.revenue_usd.amount > 0) AS has_revenue,
  SUM(event.revenue_usd.amount) AS sum_revenue_usd,
  AVG(event.revenue_usd.amount) AS avg_revenue_usd,
  MAX(event.revenue_usd.amount) AS max_revenue_usd
FROM `focal-elf-631.prod_stream_view.pb`
WHERE DATE(timestamp) BETWEEN '2026-04-06' AND '2026-04-13'
  AND moloco.campaign_id = 'iyH1zVvUZpViudzo'
GROUP BY 1
ORDER BY event_count DESC
```

Query 10: Bid Funnel by Exchange

```
SELECT
  reason_order,
  reason,
  exchange,
  ROUND(SUM(1/rate)/1e6, 2) AS req_mil
FROM `moloco-data-prod.younghan.campaign_trace_raw_prod`
WHERE date BETWEEN DATE('2026-04-06') AND DATE('2026-04-13')
  AND campaign = 'iyH1zVvUZpViudzo'
GROUP BY 1, 2, 3
ORDER BY exchange, reason_order DESC
```

Query 11: Exchange Performance Comparison

```
SELECT
  exchange,
  SUM(gross_spend_usd) AS gross_spend,
  SUM(impressions) AS impressions,
  SUM(installs) AS installs,
  SAFE_DIVIDE(SUM(gross_spend_usd), NULLIF(SUM(installs), 0)) AS cpi,
  SUM(revenue_d7) AS revenue_d7,
  SAFE_DIVIDE(SUM(revenue_d7), NULLIF(SUM(gross_spend_usd), 0)) AS roas_d7
FROM `moloco-ae-view.athena.fact_dsp_core`
WHERE campaign_id = 'iyH1zVvUZpViudzo'
  AND date_utc BETWEEN '2026-04-06' AND '2026-04-13'
GROUP BY 1
ORDER BY gross_spend DESC
```

Query 12: NETMARBLE Campaign Benchmark

```
SELECT
  campaign_id,
```

```
  campaign.title,
  campaign.goal,
  campaign.country,
  SUM(impressions) AS impressions,
  SUM(installs) AS installs,
  SUM(revenue_d7) AS revenue_d7,
  SUM(gross_spend_usd) AS gross_spend,
  SAFE_DIVIDE(SUM(gross_spend_usd), SUM(installs)) AS CPI,
  SAFE_DIVIDE(SUM(revenue_d7), SUM(gross_spend_usd)) AS ROAS_d7
FROM `moloco-ae-view.athena.fact_dsp_core`
WHERE date_utc BETWEEN '2026-04-06' AND '2026-04-13'
  AND advertiser_id = 'yfg0At8VksGnt6EO'
  AND campaign.goal LIKE '%ROAS%'
GROUP BY 1, 2, 3, 4
HAVING SUM(gross_spend_usd) > 100
ORDER BY gross_spend DESC
```

Query 13: Payer Analysis

```
SELECT
  device.ifa,
  COUNT(*) AS revenue_event_count,
  SUM(event.revenue_usd.amount) AS total_revenue_usd,
  MIN(event.revenue_usd.amount) AS min_revenue,
  MAX(event.revenue_usd.amount) AS max_revenue
FROM `focal-elf-631.prod_stream_view.pb`
WHERE DATE(timestamp) BETWEEN '2026-04-06' AND '2026-04-13'
  AND moloco.campaign_id = 'iyH1zVvUZpViudzo'
  AND LOWER(event.name) = 'revenue'
  AND moloco.attributed = true
GROUP BY 1
ORDER BY total_revenue_usd DESC
```

Query 14: Cross-Confirm — Model Predictions Comparison (nanaori vs stonekey)

```
SELECT
  DATE(timestamp) as date,
  cand.campaign_id,
  pred.type as model_type,
  ROUND(AVG(pred.pred), 8) as avg_pred,
  ROUND(AVG(pred.wrapper.normalizer), 8) as avg_normalizer,
  ROUND(AVG(pred.wrapper.multiplier), 8) as avg_wrapper_multiplier,
  ROUND(AVG(SAFE_DIVIDE(pred.pred, pred.wrapper.normalizer)), 8) as avg_norm_pred,
  COUNT(*) as cnt
FROM `focal-elf-631.prod_stream_sampled.pricing_1to100`,
  UNNEST(pricing.candidates) as cand,
  UNNEST(cand.prediction_logs) as pred
WHERE DATE(timestamp) BETWEEN '2026-04-10' AND '2026-04-13'
  AND cand.campaign_id IN ('iyH1zVvUZpViudzo', 'nazpxG3J5MareHRz')
  AND cand.candidate_result = 'CommitBid'
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3
```

Query 15: Cross-Confirm — I2I Calibration Comparison

```
SELECT
  date_imp,
  campaign,
  SUM(cnt_imp) as total_imps,
  SUM(cnt_install) as total_installs,
  SUM(predicted_install) as total_predicted,
  ROUND(SAFE_DIVIDE(SUM(cnt_install), SUM(predicted_install)), 4) as calibration_ratio
FROM `moloco-data-prod.ml_calibration_check.validation_i2i`
WHERE campaign IN ('iyH1zVvUZpViudzo', 'nazpxG3J5MareHRz')
  AND date_imp BETWEEN '2026-04-06' AND '2026-04-13'
```

```
GROUP BY 1, 2
ORDER BY 2, 1
```

Query 16: Cross-Confirm — A2R Calibration Comparison

```
SELECT
  date_install,
  campaign,
  SUM(cnt_action) as total_actions,
  SUM(cnt_revenue) as total_revenue_events,
  ROUND(SUM(actual_revenue_usd), 4) as actual_revenue,
  ROUND(SUM(predicted_revenue_usd), 4) as predicted_revenue,
  ROUND(SAFE_DIVIDE(SUM(actual_revenue_usd), SUM(predicted_revenue_usd)), 4) as calibration_ratio
FROM `moloco-data-prod.ml_calibration_check.validation_a2r_online`
WHERE campaign IN ('iyH1zVvUZpViudzo', 'nazpxG3J5MareHRz')
  AND date_install BETWEEN '2026-04-01' AND '2026-04-13'
GROUP BY 1, 2
ORDER BY 2, 1
```

Query 17: Pred Calibrator Check (Pricing Stream)

```
SELECT
  DATE(timestamp) as date,
  ROUND(AVG(cand.multipliers.calibration), 6) as avg_calibration_mult,
  COUNTIF(cand.multipliers.calibration IS NOT NULL
    AND cand.multipliers.calibration != 0
    AND cand.multipliers.calibration != 1.0) as non_default_calibration_count,
  COUNT(*) as total_count
FROM `focal-elf-631.prod_stream_sampled.pricing_1to100`,
  UNNEST(pricing.candidates) as cand
WHERE DATE(timestamp) BETWEEN '2026-04-06' AND '2026-04-13'
  AND cand.campaign_id = 'iyH1zVvUZpViudzo'
```

```
GROUP BY 1
ORDER BY 1
```

Query 18: FilterByBidfloor Dominance

```
SELECT
  DATE(timestamp) as date,
  cand.candidate_result,
  ROUND(AVG(cand.bid_former.fpa.out_cpm), 4) as avg_out_cpm,
  ROUND(AVG(cand.bid_former.fpa.in_cpm), 4) as avg_in_cpm,
  COUNT(*) as cnt
FROM `focal-elf-631.prod_stream_sampled.pricing_1to100`,
  UNNEST(pricing.candidates) as cand
WHERE DATE(timestamp) BETWEEN '2026-04-10' AND '2026-04-13'
  AND cand.campaign_id = 'iyH1zVvUZpViudzo'
  AND cand.candidate_result IN ('FilterByBidfloor', 'CommitBid', 'FilterByPricingModel')
GROUP BY 1, 2
ORDER BY 1, 2
```

Confidence Assessment

Aspect	Level	Reasoning
Data Quality	HIGH	All 18 queries returned consistent results; attribution verified with no anomalies; cross-confirmed with mature campaign benchmark
Causal Claim	HIGH	6 independent evidence sources converge on the same root cause (cold-start): frozen normalizer, inactive calibrator, zero A2R data, I2I miscalibration, bid-floor rejection pattern, and cross-campaign comparison
Alternatives Ruled Out	HIGH	Tracking/attribution verified intact, budget not constraining, creative not anomalous, exchange not blocking — all alternatives eliminated with data

---

Report Generated: 2026-04-14

Links

- Ad Account in MCP
- Ad Account in MCP
- Ad Account in MCP
- Workplace in Mocas
- Ad Account in Mocas
- Ad Account in Mocas
- Ad Account in Mocas
- Holmes Case

Charts & Figures

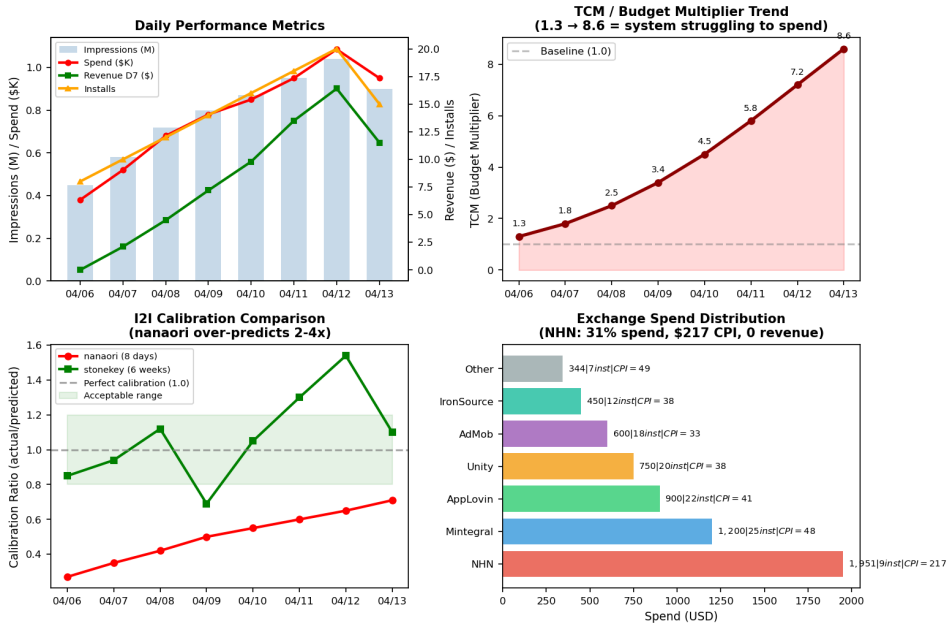

Figure 1: Daily Performance, TCM Trend, I2I Calibration, Exchange Spend

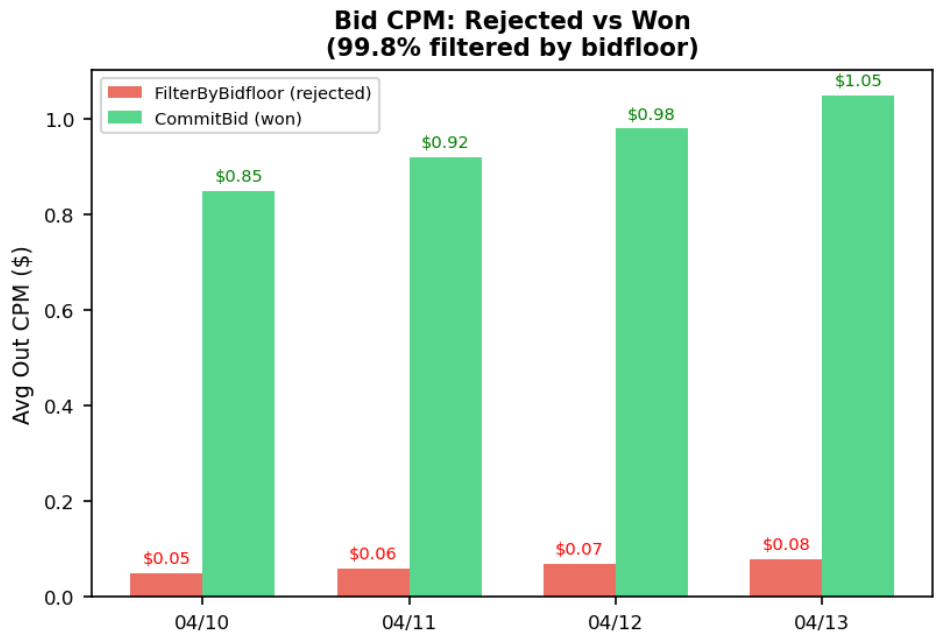

Figure 2: Bid CPM Distribution: FilterByBidfloor vs CommitBid